

SAT-Web Manual

J. Madgwick

1st June 2025

1 Introduction

SAT-Web is a web-based educational tool for explaining SAT solving, and the first tool of its kind which can be used entirely from a web-browser. It provides visualisations with search trees and variable interaction graphs, and includes a DPLL solver with tracing. This manual briefly explains the features of SAT-Web (Section 2) and how to use it (Section 3).

2 Features

SAT-Web is a single page application (SPA) style interactive web app where all components are visible on the screen at the same time. A panel based layout is used with separate spaces set aside for each feature. No downloads or runtimes are required to use SAT-Web, only a web-browser and access to a URL where the app is hosted.

An example of the SAT-Web interface is shown in Figure 1. SAT-Web has the following components: CNF input area, example SAT problem selection, solver control panel, log for solver tracing, instance representation using mathematical notation, solver information panel, search tree graph visualisation, and variable interaction graph visualisation.

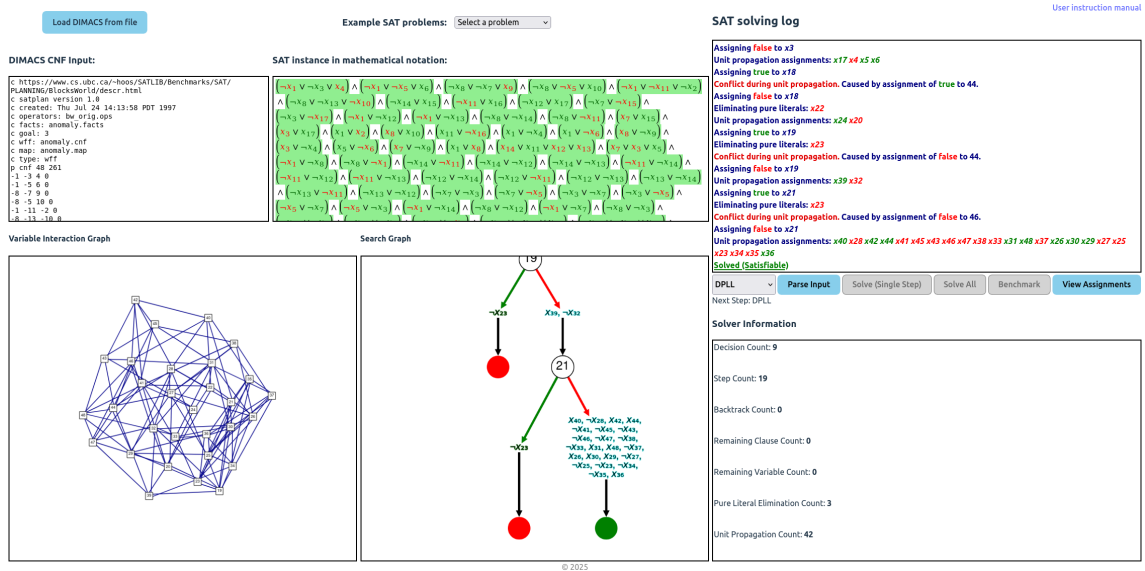


Figure 1: The SAT-Web interface after an example problem has been loaded and solved successfully.

2.1 CNF input and Mathematical Notation display

The *CNF input* area allows you to manually input a SAT problem (using DIMACS format). You can also loading problems directly from a local file on your computer using the *Load DIMACS from file* button. The *example SAT problem* dropdown menu allows for example problems to be loaded

directly into the input box. These problems range from very simple (solvable by hand), to some well known more complex problems which cannot be solved by backtracking solvers in a reasonable time. The *mathematical notation panel* (Figure 2) uses colour to indicate the currently assigned boolean value of literals and clauses within the current problem. It is only shown once the *Parse Input* button has been successfully used to read a problem from the input area, see Section 3.

SAT instance in mathematical notation:

$\neg x_{26} \vee \neg x_{22}$	$\wedge$	$\neg x_{24} \vee \neg x_{18}$	$\wedge$	$\neg x_{21} \vee \neg x_{17}$	$\wedge$	$\neg x_{25} \vee \neg x_{17}$	$\wedge$	$\neg x_{25} \vee \neg x_{21}$	$\wedge$	$\neg x_{19} \vee \neg x_{13}$	$\wedge$
$\neg x_{25} \vee \neg x_{13}$	$\wedge$	$\neg x_{25} \vee \neg x_{19}$	$\wedge$	$\neg x_{16} \vee \neg x_{12}$	$\wedge$	$\neg x_{20} \vee \neg x_{12}$	$\wedge$	$\neg x_{20} \vee \neg x_{16}$	$\wedge$	$\neg x_{24} \vee \neg x_{12}$	$\wedge$
$\neg x_{24} \vee \neg x_{16}$	$\wedge$	$\neg x_{24} \vee \neg x_{20}$	$\wedge$	$\neg x_{14} \vee \neg x_8$	$\wedge$	$\neg x_{20} \vee \neg x_8$	$\wedge$	$\neg x_{20} \vee \neg x_{14}$	$\wedge$	$\neg x_{26} \vee \neg x_8$	$\wedge$
$\neg x_{26} \vee \neg x_{14}$	$\wedge$	$\neg x_{26} \vee \neg x_{20}$	$\wedge$	$\neg x_{11} \vee \neg x_7$	$\wedge$	$\neg x_{15} \vee \neg x_7$	$\wedge$	$\neg x_{15} \vee \neg x_{11}$	$\wedge$	$\neg x_{19} \vee \neg x_7$	$\wedge$
$\neg x_{19} \vee \neg x_{11}$	$\wedge$	$\neg x_{19} \vee \neg x_{15}$	$\wedge$	$\neg x_{23} \vee \neg x_7$	$\wedge$	$\neg x_{23} \vee \neg x_{11}$	$\wedge$	$\neg x_{23} \vee \neg x_{15}$	$\wedge$	$\neg x_{23} \vee \neg x_{19}$	$\wedge$
$\neg x_{18} \vee \neg x_6$	$\wedge$	$\neg x_{14} \vee \neg x_6$	$\wedge$	$\neg x_{14} \vee \neg x_{16}$	$\wedge$	$\neg x_{18} \vee \neg x_6$	$\wedge$	$\neg x_{18} \vee \neg x_{10}$	$\wedge$	$\neg x_{18} \vee \neg x_{14}$	$\wedge$
$\neg x_{12} \vee \neg x_6$	$\wedge$	$\neg x_9 \vee \neg x_5$	$\wedge$	$\neg x_{13} \vee \neg x_5$	$\wedge$	$\neg x_{13} \vee \neg x_9$	$\wedge$	$\neg x_{11} \vee \neg x_5$	$\wedge$	$\neg x_{17} \vee \neg x_5$	$\wedge$
$\neg x_8 \vee \neg x_4$	$\wedge$	$\neg x_{10} \vee \neg x_4$	$\wedge$	$\neg x_{16} \vee \neg x_4$	$\wedge$	$\neg x_{16} \vee \neg x_{10}$	$\wedge$	$\neg x_{22} \vee \neg x_4$	$\wedge$	$\neg x_{22} \vee \neg x_{10}$	$\wedge$

Figure 2: Mathematical notation component. Literals are coloured based on their resultant boolean value, with clauses highlighted in green if they are satisfied or red if they are not.

2.2 SAT problem visualisations

The *variable interaction graph visualisation* (Figure 3) can be zoomed, panned, and nodes in the graph can be moved around.

Variable interaction graphs show relationships between variables across all instance clauses. Each variable is represented as a node, with edges drawn between variables which exist in the same clause. They are helpful for understanding an instance's current structure, with specific shapes, clusters, and other visual patterns being seen with some SAT instances.

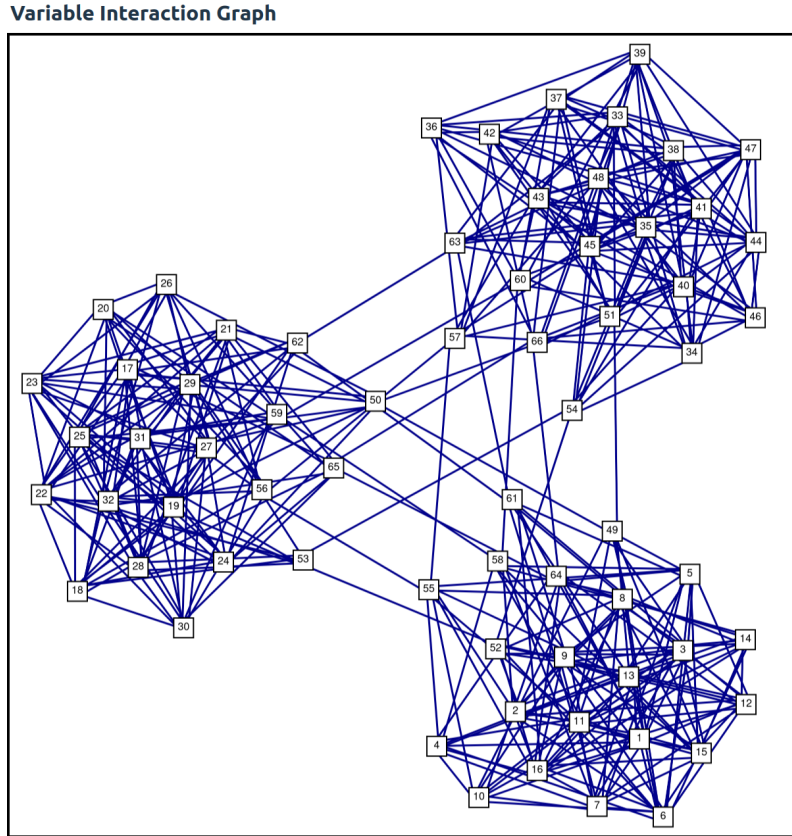


Figure 3: Variable interaction graph generated before solving has started showing the structure of a problem.

The *search graph visualisation* (Figures 4 and 5) supports the same interaction features as the variable interaction graph.

Search graphs (search trees) are a form of graph representing the sequence of variable assignments, beginning from a root node and branching downwards as variables are assigned. They are effective for intuitively visualising backtracking as moving upwards through branches in the tree.

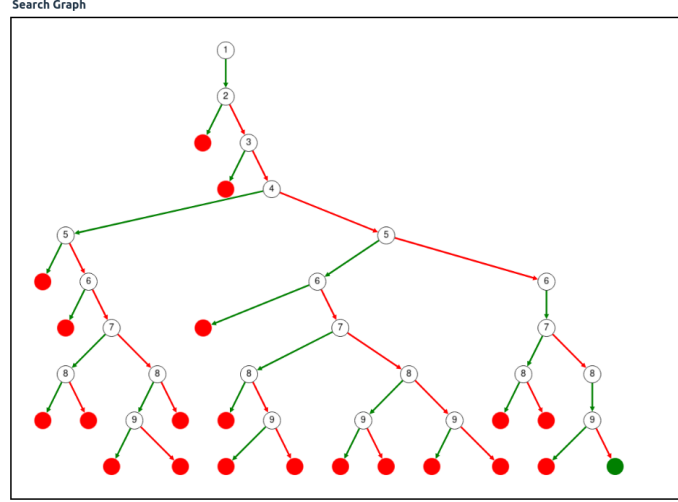


Figure 4: Search graph component showing a small problem solved using backtracking. The green node indicates that satisfiability was achieved, red nodes represent conflicts.

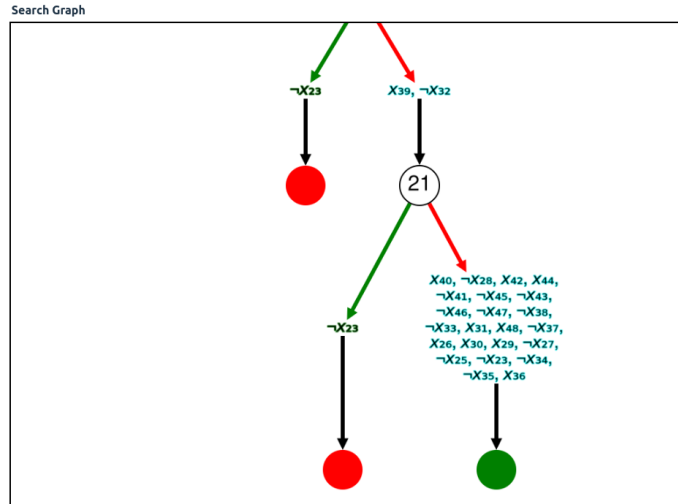


Figure 5: Search graph component showing part of the tree produced when solving a problem using DPLL. Assignments made by unit propagation have a cyan outline, with pure literal elimination assignments indicated by a green outline.

2.3 Solver log and control panel

The *solver control panel* controls all aspects of the app, such as parsing CNF input, selecting algorithms, and viewing final variable assignments. The *solver log* (Figure 6) displays traces explaining the internal operations of the solver as you step through a problem, with colour used to emphasise assignments and conflicts. The *solver information panel* displays details such as how many times backtracking has occurred and the number of assignments made by unit propagation or pure literal elimination.

SAT solving log

```
Assigning true to x28
Assigning true to x29
Conflict during unit propagation. Caused by assignment of false to 31.
Assigning false to x29
Conflict during unit propagation. Caused by assignment of false to 31.
Backtracking.
Assigning false to x28
Conflict during unit propagation. Caused by assignment of true to 46.
Backtracking.
Assigning false to x27
Conflict during unit propagation. Caused by assignment of true to 30.
Backtracking.
Assigning false to x26
Unit propagation assignments: x56 x65 x57 x66 x34 x36 x37 x40 x42 x43 x45 x48 x33 x44 x46
Assigning true to x27
Assigning true to x28
Assigning true to x29
Unit propagation assignments: x31 x32 x59 x30 x60 x39 x41 x47 x38 x35 x54 x63 x53 x62
Solved (Satisfiable)
```

Figure 6: SAT tracing log component showing entries after solving a problem.

3 How to use SAT-Web

The section explains how to load, parse, and solve a SAT problem and how to use the solver control panel.

3.1 Loading and parsing a SAT problem

Follow these steps to load and parse a problem, ready to start solving:

1. Type (or paste from the clipboard) a SAT problem into the *input area* using DIMACS format. Or load an example problem from the *example SAT problem* dropdown menu.
2. (Optional - defaults to DPLL) Select the desired SAT solver algorithm using the *algorithm selection* dropdown menu, located on the left of the *Parse Input* button.
3. Press the *Parse Input* button to process the SAT problem inputted.
4. If there is a problem parsing the input, a message box will be shown with details. If there are no problems, the mathematical notation for the problem will be populated and the remaining solver control panel buttons enabled.

3.2 Solving a SAT problem

Once a SAT problem has been loaded and parsed successfully, you can work through the problem using the chosen SAT solver algorithm. This is done by using the *Solve (Single Step)* button multiple times or by using the *Solve All* button once. The *Solve (Single Step)* button performs a single step of the chosen solver algorithm, updating all visualisations and the solver log afterwards. The *Solve All* button performs as many steps as are necessary to solve the problem, or determine that it is not possible to solve.

After using *Solve All*, only the search graph and solving log are updated. Note that for large problems there may be a delay between using the button and the interface updating. **Problems which cannot easily be solved with the available algorithms currently cause SAT-Web to freeze**, avoid inputting these problems.

The *View Assignments* button shows the current set of assignments; it can be used at any point in the solving process. The *Benchmark* button performs the same function as *Solve All*, except that once solving is complete a message is shown detailing the time taken to solve the problem, along with the final assignments.